

พจนานุกรมข้อมูล (Data Dictionary)

ระบบประกันรายได้ SGI / SBP Mall — ตารางที่ต้องสร้างใหม่ทั้งหมด

20 ตาราง · 251 คอลัมน์ · อธิบายครบทุกคอลัมน์

ขอบเขต	20 ตารางใหม่ (โซน A 8 · B 9 · C 3) · 251 คอลัมน์
ตอบคำถาม	ตารางนี้มาจากไหน · ไขทำอะไร · ทำไมต้องมี · แต่ละคอลัมน์เก็บอะไรไว้ทำอะไร
ไม่รวม	ตารางของระบบ SBP เดิมที่ SGI แคว่อ่าน — fcs_qssi_score · store/mas_store/sevenshop · common_code · mas_param · business_user · email_template · sps_store.workflow_* (13 ตาราง)
อ่านคู่กับ	LLDD-Database (DDL · index · transaction) · database.md (ตารางต้นทางตอน migrate)
ที่มาของเนื้อหา	tools/lldd_db_dictionary.py — รายชื่อคอลัมน์ดึงสดจาก DDL ทุกครั้งที่ build

สารบัญ

โซน	ตาราง	ทำหน้าที่อะไร	คอลัมน์
A	sgi_fgi_impact_processes	hub ของ "หนึ่งรอบชดเชย" (ร้าน × งวดเดือน)	19
A	sgi_fgi_impact_stores	คู่ "ร้านถูกระทบ × ร้านเปิดใหม่" ของแต่ละรอบ	16
A	sgi_fgi_impact_sales_summaries	สรุปยอดขายก่อน/หลังร้านใหม่เปิด (หัวของยอดขาย)	9
A	sgi_fgi_new_store_compensations	ค่าชดเชยฝั่งร้านเปิดใหม่ รายงวด (ลูกของค่าชดเชยฝั่งร้านถูกระทบ)	12
A	sgi_sales_transactions	ยอดขายรายวันจาก IAS (ลูกของตารางสรุป)	10
A	sgi_fgi_impact_competitors	ร้านคู่แข่งที่ถูกระทบ (ระดับ pipeline)	14
A	sgi_fgi_impact_compensations	ยอดชดเชยรายงวดของแต่ละรอบ	18
A	sgi_interface_transactions	สมุดกลางของการรับ-ส่งข้อมูลกับระบบภายนอก (tracking + transactional outbox)	26
B	sgi_compensation_documents	เอกสารประกันรายได้ (หัวเอกสาร) — ตารางกลางของทั้งโซน B	26
B	sgi_document_new_stores	ร้านเปิดใหม่ในเอกสาร + การปันส่วน %ชดเชย	9
B	sgi_document_competitors	ร้านคู่แข่งในเอกสาร (ระดับสาขา)	18
B	sgi_document_external_factors	ปัจจัยภายนอกที่ยกมาในเอกสาร	8
B	sgi_consideration_logs	ประวัติการพิจารณาทุกชั้น (ใครตัดสินใจอะไร เมื่อไร)	9
B	sgi_document_attachments	ไฟล์แนบของเอกสาร (เก็บเฉพาะ metadata ตัวไฟล์อยู่บน S3)	16
B	sgi_compensation_histories	ประวัติการชดเชยรายร้าน (ข้ามเอกสาร)	8
B	sgi_document_cost_details	ยอดชดเชยแยกรายเดือน × รายร้านเปิดใหม่	10
B	sgi_document_running_numbers	ตัวนับเลขเอกสารต่อปี	4
C	sgi_impacted_stores	ทะเบียนร้านที่ถูกระทบ (ร้าน SP ที่อยู่ในขอบเขตประกันรายได้)	8
C	sgi_external_factors	master ปัจจัยภายนอก	5
C	sgi_competitors	master แบนด์คู่แข่ง	6

โซน A FGI/FCS Impact Pipeline และ interface ภายนอก — 8 ตาราง

sgi_fgi_impact_processes

19 คอลัมน์

hub ของ "หนึ่งรอบชดเชย" (ร้าน × งวดเดือน)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE_ON_PROCESS (PK IMPACT_PROCESS_ID · sequence SEQ_FGI_IMPACT_PROCESS)
ใช้ทำอะไร	เป็น แกมมาของทุกอย่างในรอบชดเชยหนึ่งรอบ — 1 แกมมา = ร้านหนึ่งร้าน ในงวดหนึ่งงวด ตารางลูกทั้งหมดในโซน A ห้อยจาก id ของตารางนี้
ทำไมต้องมี	ถ้าไม่มีแกมมา ข้อมูลรายรอบ (ร้านเปิดใหม่ · ยอดขาย · คู่แข่ง · ยอดชดเชย) จะไม่มีอะไรมัดรวมกัน และจะตอบไม่ได้ว่า "รอบนี้ประมวลถึงไหนแล้ว" · id ของตารางนี้คือต้นทางของ data spine <code>impact_process_id → doc_no → transaction_id → approver_id</code>
ใครเขียน	Job 2 (สร้างรอบ) · Job 4/5 (อัปเดตสถานะ) · Job 8b (ตั้ง workflow_generation_status)
ใครอ่าน	Job 6/7/8/9 · GET /sgi/workflow/summary · รายงานสรุปสถานะ

กติกาส่ที่ต้องรู้ก่อนเขียนโค้ด

- UNIQUE (impacted_store_code, impact_month) — ร้านหนึ่งร้านมีได้รอบเดียวต่อหนึ่งงวด กันสร้างซ้ำตอน rerun
- workflow_generation_status อยู่ที่ตารางนี้ ที่เดียว — ห้ามมีธงซ้ำที่ sgi_fgi_impact_stores

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impacted_store_code	VARCHAR(5)	รหัสร้านที่ถูกกระทบในรอบนี้	FK ไป sgi_impacted_stores · ครั้งหนึ่งของ business key
impact_month	CHAR(7)	งวดเดือนที่กระทบ รูปแบบ YYYY-MM (ค.ศ.)	อีกครึ่งของ business key · ใช้เป็นตัวกรองหลักของทุกรายงาน
impact_year	INTEGER	ปี ค.ศ. ของงวด แยกออกมาเป็นตัวเลข	มีไว้เพื่อ กรอง/จัดกลุ่มรายปีโดยไม่ต้อง substring impact_month ซึ่งทำให้ index ใช้ไม่ได้
process_status	VARCHAR(30)	สถานะการประมวลผลของรอบ (ขั้นตอน pipeline ถึงไหนแล้ว)	ใช้ให้ job ที่รันทีหลังรู้ว่างานก่อนหน้าเสร็จหรือยัง และใช้ตอน rerun ว่าจะข้ามแกมมาไหน
action_status	VARCHAR(30)	สถานะการตัดสินใจของรอบ (Y = ดำเนินการต่อ · W = รอ · N = ไม่เข้าเกณฑ์)	เป็น gate ว่ารอบนี้จะถูกสร้างเป็นเอกสารหรือไม่ — Job 8 อ่านค่านี้
last_compensation_amount	NUMERIC(14,2)	ยอดชดเชยของรอบล่าสุดของร้านนี้	ใช้เทียบว่ายอดงวดนี้เป็น 0 ติดกันกี่เดือน เพื่อบังคับกฎ "เดือนที่ 4 หยุดชดเชย"
workflow_generation_status	CHAR(1)	ผลของ Gen Flow Gate: W = ข้อมูลยังไม่พร้อม (ลองใหม่ได้) · Y = เปิด workflow สำเร็จ · N = ไม่เข้าเกณฑ์ถาวร	คอลัมน์เดียวที่ Job 8b ใช้ตัดสินใจและกันเปิด workflow ซ้ำ · W ต่างจาก N ตรงที่ W จะถูกหยิบมาลองใหม่รอบหน้า ส่วน N จบถาวร
last_compensate_seq	INTEGER	ลำดับรอบชดเชยล่าสุด (นับรวมทุกครั้ง)	ใช้ต่อเลขลำดับให้แกมมาใหม่ใน sgi_fgi_impact_compensations

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
last_compensate_seq_no	INTEGER	เลขลำดับย่อยของรอบล่าสุด	ใช้คู่กับ last_compensate_seq ตอนรอบเดียวมีหลายวงย่อย
start_compensate_month	CHAR(7)	งวดเดือนแรกที่เริ่มชดเชยร้านนี้ (YYYY-MM ค.ศ.)	ใช้นับว่าตอนนี้เป็น "เดือนที่เท่าไรของการชดเชย" ซึ่งเป็นหัวใจของกฎเดือน 1-3 / เดือน 4
start_compensate_year	INTEGER	ปี ค.ศ. ของงวดแรก	แยกไว้เพื่อ query รายปีโดยไม่ substring
end_compensate_month	CHAR(7)	งวดเดือนสุดท้ายที่ชดเชย	ตั้งเมื่อรอบจบ (หยุดชดเชย) · ใช้ปีรอบไม่ให้ job หยิบมาทำอีก
end_compensate_year	INTEGER	ปี ค.ศ. ของงวดสุดท้าย	คู่กับ end_compensate_month
flag_action	CHAR(1)	ธงบอกว่ารอบนี้ถูกดำเนินการแล้วหรือยัง (Y/N)	ตัวกัน job หยิบแถวเดิมซ้ำภายในรอบการรันเดียวกัน
datasource	VARCHAR(5)	ระบบต้นทางของแถว เช่น ALM (ALLMAP) · FGI	ใช้แยกว่าข้อมูลมาจาก pipeline อัตโนมัติหรือคีย์มือ ตอนสืบหาสาเหตุข้อมูลผิด
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้แอปเขียนทับ
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้ไขเมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_impact_stores

16 คอลัมน์

คือ "ร้านถูกระทบ × ร้านเปิดใหม่" ของแต่ละรอบ

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE (PK IMPACT_STORE_ID · business key STORECODE_I + MONTH + YEAR)
ใช้ทำอะไร	เก็บว่ารอบนี้ร้าน SP ถูกระทบจาก ร้านเปิดใหม่อันไหนบ้าง พร้อมระยะทางและ %ชดเชยที่ปันให้แต่ละร้าน
ทำไมต้องมี	ร้านหนึ่งร้านอาจถูกระทบจากร้านเปิดใหม่หลายร้านพร้อมกัน · ยอดชดเชยต้อง ปันส่วนตาม % ให้ครบ 100% พอดี ตารางนี้คือที่เก็บการปันส่วนนั้น
ใครเขียน	Job 2 (สร้างคู่จาก ALLMAP) · Job 4/5 (เติม %/ยอด) · Job 9 (sync ไปเอกสาร)
ใครอ่าน	Job 6/8/9 · sgi_document_new_stores (ปลายทางที่ sync ไป)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ผลรวม forecast_compensate_percent ของทุกแถวใน impact_process_id เดียวกัน ต้องเท่ากับ 100 พอดี — ตรวจก่อน commit เสมอ
- ▲ ตารางนี้ ไม่มีคอลัมน์ impact_year (มีแต่ impact_month) — เคยมีคนเดาว่ามีแล้วเขียน SQL พัง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิม เปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
impact_process_id	BIGINT	FK ไปแถวแม่ในรอบเดียวกัน	มัดทุกคู่เข้ากับรอบชดเชยรอบเดียว
impacted_store_code	VARCHAR(5)	รหัสร้าน SP ที่ถูกระทบ	เก็บซ้ำจากแถวแม่เพื่อให้ query ระดับคู่ไม่ต้อง join ขึ้นไป
new_store_code	VARCHAR(5)	รหัสร้าน 7-Eleven ที่เปิดใหม่และเป็นต้นเหตุ	อีกครึ่งของคู่ · ใช้ join ไป master ร้านของระบบเดิมเพื่อเอาชื่อ/พิกัด
impact_month	CHAR(7)	งวดเดือนของคู่ (YYYY-MM ค.ศ.)	ใช้ยืนยันว่าแถวลูกอยู่งวดเดียวกับแถวแม่
distance_km	NUMERIC(8,3)	ระยะทางจากร้านเปิดใหม่ถึงร้าน SP (กม. ทศนิยม 3 ตำแหน่ง)	ใช้ตรวจเกณฑ์รัศมี 1 กม. (กทม./ปริมณฑล) · 2 กม. (ต่างจังหวัด) — เกินรัศมี = ไม่เข้าเกณฑ์
verify_status	CHAR(1)	ผลการตรวจสอบคู่ (Y/N/ว่าง)	ระบุว่าคู่นี้ผ่านการตรวจแล้วหรือยัง ก่อนเอาไปคำนวณต่อ
created_by	VARCHAR(10)	ผู้สร้างแถว — x-user-id ของผู้ใช้จริง หรือชื่อ job (เช่น JOB-8) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ติดตั้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
updated_by	VARCHAR(10)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สับย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ
sales_request_status	CHAR(1)	สถานะการขอข้อมูลยอดขายจาก IAS: W รอส่ง · P ส่งแล้วรอผล · Y ได้ผลแล้ว · E ผิดพลาด	เป็นตัวคุมลำดับ Job 4 (ส่งคำขอ) → Job 5 (รับผล) · เพิ่มค่า E เมื่อ 2026-09-02 ปิดช่องว่าง G1 เพราะเดิมไม่มีที่เก็บผล DENY
forecast_compensate_percent	NUMERIC(7,4)	%ชดเชยที่ป้อนให้ร้านเปิดใหม่อันนี้ (ค่าคาดการณ์)	ตัวคูณหลักของการคำนวณยอด · ผลรวมทุกแถวต้อง = 100
adjust_compensate_percent	NUMERIC(7,4)	%ชดเชยหลังปรับแก้โดยผู้ใช้	ใช้แทนค่า forecast เมื่อมีการปรับ · nullable = ยังไม่ปรับ
forecast_compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยที่คาดการณ์สำหรับคู่นี้	= ยอดรวมของรอบ × forecast_compensate_percent
adjust_compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยหลังปรับ	ค่าที่ใช้จริงเมื่อมีการปรับแก้
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้ไขเมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_impact_sales_summaries

9 คอลัมน์

สรุปยอดขายก่อน/หลังร้านใหม่เปิด (หัวของยอดขาย)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE_SALES (key STORECODE_I + MONTH + YEAR)
ใช้ทำอะไร	เก็บ ผลสรุปการเปรียบเทียบยอดขายก่อนและหลังร้านใหม่เปิด — อัตราเติบโตก่อน/หลังและผลต่าง ซึ่งเป็นตัวตัดสินว่าร้านนี้ "ยอดตกจริง" หรือไม่
ทำไมต้องมี	เกณฑ์ธุรกิจหลักคือ growth ต่างกัน $\leq -10\%$ ถึงจะเข้าข่ายชดเชย · ต้องมีที่เก็บผลค่านวณนี้แยกจากยอดขายรายวัน เพราะ API และรายงานอ่านค่าสรุปบ่อยกว่ารายวันมาก
ใครเขียน	Job 5 (ImportImpactSaleFromIAS) — คำนวณจากยอดขายวันที่รับมา
ใครอ่าน	Job 8 (สร้างเอกสาร) · Job 8b (Gen Flow Gate) · หน้าเอกสาร (กราฟ/ตารางยอดขาย)

กติกาก่อนที่ควรเขียนโค้ด

- 1 แถวต่อ 1 impact_process_id — เป็นหัวของ sgi_sales_transactions แบบ 1:N

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มดยอดขายเข้ากับรอบ
total_working_days	INTEGER	จำนวนวันทำการที่มีข้อมูลยอดขายจริง	เกณฑ์คือ ต้องครบ 60 วัน · ไม่ครบ = ร้านขึ้นธงแดง "ยอดขายไม่ครบ 60 วัน" และเข้าเงื่อนไข pre-accept
growth_rate_before	NUMERIC(9,4)	อัตราเติบโตโดยยอดขาย ก่อน ร้านใหม่เปิด (%)	ฐานเปรียบเทียบ
growth_rate_after	NUMERIC(9,4)	อัตราเติบโตโดยยอดขาย หลัง ร้านใหม่เปิด (%)	ค่าที่เอามาเทียบกับฐาน
growth_rate_diff	NUMERIC(9,4)	ผลต่าง = after - before (%)	ตัวตัดสินเกณฑ์หลัก: ต้อง ≤ -10 ถึงเข้าข่ายชดเชย · ถ้าเป็น NULL แปลว่าข้อมูลยังไม่พร้อม Gate จะคง W ไว้
sales_status	CHAR(1)	ระบุว่าข้อมูลยอดขายพร้อมใช้หรือยัง (Y/N)	เป็นเงื่อนไขหนึ่งของ Gen Flow Gate — ค่าอื่นที่ไม่ใช่ Y/N ให้คง W และคืน 422
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้ไขเมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_fgi_new_store_compensations

12 คอลัมน์

ค่าชดเชยฝั่งร้านเปิดใหม่ รายงวด (ลูกของค่าชดเชยฝั่งร้านถูกระทบ)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_NEW_STORE_COMPENSATE (23,628 แถวจริง)
ใช้ทำอะไร	แตกยอดชดเชยของงวดหนึ่ง ออกเป็น รายร้านเปิดใหม่ พร้อมสัดส่วน %ที่แต่ละร้านรับผิดชอบ
ทำไมต้องมี	ร้านถูกระทบหนึ่งร้านอาจโดนร้านใหม่หลายสาขาพร้อมกัน · เงินชดเชยต้องกระจายตามสัดส่วน และ ผลรวม %ของทุกร้านใหม่ในงวดเดียวต้องได้ 100% — ถ้าไม่แตกเป็นแถว ตรวจสอบดูไม่ได้
ใครเขียน	Job 6 — insert ตอนเปิดรอบชดเชยใหม่ (mutation ที่ 6)
ใครอ่าน	Job 6 — ประกอบ payload ส่ง STA · หน้าจอเอกสาร (ตารางแบ่ง %ชดเชย)

กติกาก่อนที่ควรรู้ก่อนเขียนโค้ด

- 1 แถว = 1 (งวด × ร้านเปิดใหม่) — ไม่ใช่ 1 แถวต่อคู่ร้าน · คู่ร้านเดิมได้แถวใหม่ทุกงวดที่ยังชดเชยต่อเนื่อง
- ห้ามย้ายข้อมูลนี้ไปไว้บน sgi_fgi_impact_stores — ตารางนั้นไม่มีมิติงวด งวดใหม่จะทับงวดเก่าแล้วตรวจสอบย้อนหลังไม่ได้ว่างวดไหนจ่ายเท่าไร
- ผลรวม forecast_percent ของทุกแถวที่มี impact_compensation_id เดียวกัน ต้องเท่ากับ 100

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_compensation_id	BIGINT	FK ไปค่าชดเชยฝั่ง I ของงวดนั้น	มดยอดรายร้านใหม่เข้ากับงวด · ON DELETE CASCADE เพราะลูกไม่มีความหมายถ้าไม่มีงวด
impact_store_id	BIGINT	FK ไปแถวคู่ร้าน (ร้านถูกระทบ ↔ ร้านเปิดใหม่)	ORA IMPACT_STORE_ID · ตามรอยกลับไปหาต้นเรื่องว่าคู่นี้มาจาก import รอบไหน
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่ 5 หลัก	ฟิลด์ที่ 2 ของ payload ที่ส่ง STA
forecast_amount	NUMERIC(14,2)	ยอดชดเชยที่ประมาณการไว้ (บาท)	ค่าตั้งต้นก่อนผู้พิจารณาปรับ
forecast_percent	NUMERIC(7,4)	สัดส่วน %ที่ร้านใหม่นี้รับผิดชอบ (ประมาณการ)	รวมทุกแถวในงวดเดียวต้องได้ 100
adjust_amount	NUMERIC(14,2)	ยอดชดเชยหลังปรับ (บาท)	ค่าที่ผู้พิจารณาแก้ — วางแปลว่ายังไม่ถูกปรับ
adjust_percent	NUMERIC(7,4)	สัดส่วน %หลังปรับ	รวมทุกแถวในงวดเดียวต้องได้ 100 เช่นกัน
created_by	VARCHAR(100)	ผู้สร้างแถว — x-user-id ของผู้ใช้จริง หรือชื่อ job (เช่น JOB-8) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ตัดทิ้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชิ้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ
updated_by	VARCHAR(100)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_sales_transactions

10 คอลัมน์

ยอดขายรายวันจาก IAS (ลูกของตารางสรุป)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE_SALES_TRN (key เดียวกับหัว + SEQ) · ข้อมูลใหม่มาจากไฟล์ AMS06001I ที่ IAS/MIS วางบน EAI S3
ใช้ทำอะไร	เก็บ ยอดขายรายวัน ของร้านในช่วงเปรียบเทียบ แบ่งเป็น 4 หน้าต่าง × 15 วัน รอบวันที่ร้านใหม่เปิด
ทำไมต้องมี	ตัวเลขสรุป (growth) ต้องพิสูจน์ย้อนได้ว่ามาจากยอดวันไหนบ้าง · และหน้าจอเอกสารต้องโชว์ไหมไลน์ยอดขายให้ผู้พิจารณาดูก่อนตัดสินใจ
ใครเขียน	Job 5 — insert ทั้งหมดหลังจากอ่านไฟล์จาก S3
ใครอ่าน	GET /sgi/document/{docNo}/sales (ไหมไลน์ยอดขาย) · การคำนวณ growth ของ Job 5

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ไม่ผูกกับ impact_process_id โดยตรง — ผูกกับ sales_summary_id เท่านั้น เพื่อไม่ให้มี FK ซ้ำสองทาง
- ไฟล์ต้นทางเป็น windows-874 และปี พ.ศ. — ต้องแปลงเป็น UTF-8 และ ค.ศ. ตอนอ่านไฟล์ ห้ามเก็บ พ.ศ. ลงตารางนี้

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
sales_summary_id	BIGINT	FK ไปห้ายอดขาย	มียอดรายวันเข้ากับรอบผ่านหัว
txn_date	DATE	วันที่ของยอดขายแถวนี้ (ค.ศ.)	แกน X ของกราฟยอดขาย · ใช้จัดเรียงและแบ่งหน้าต่าง
window_no	SMALLINT	หมายเลขหน้าต่างเปรียบเทียบ 1–4 (แต่ละหน้าต่าง 15 วัน)	หน้าต่างก่อน/หลังวันร้านใหม่เปิด · ใช้จัดกลุ่มตอนคำนวณ growth ก่อน-หลัง
seq	SMALLINT	ตำแหน่งวันภายในหน้าต่าง 1–15 (คอลัมน์ SEQ ของตารางเดิม)	ใช้จับคู่วันเดียวกันระหว่างปีก่อนกับปีนี้ ตอนคำนวณ growth — ตารางเดิมเป็นตารางกว้างจึงจับคู่ได้ในแถวเดียว ตารางใหม่แคบจึงต้องมีคอลัมน์นี้ join
sales_amount	NUMERIC(14,2)	ยอดขายของวันนั้น (บาท)	ตัวตั้งของการคำนวณทั้งหมด
sales_diff	NUMERIC(14,2)	เปอร์เซ็นต์ที่ยอดขายวันนั้นเบี่ยงจากค่าเฉลี่ยของหน้าต่างตัวเอง = $\text{ROUND}((\text{ยอดขายวันนั้น} - \text{เฉลี่ยหน้าต่าง}) / \text{ยอดขายวันนั้น} \times 100)$	ใช้หา outlier — หน่วยเป็นเปอร์เซ็นต์ ไม่ใช่บาท และหารด้วยยอดของวันนั้นเอง ไม่ใช่หารด้วยค่าเฉลี่ย (สูตรของระบบเดิม ImportJdbc.java:544)
is_outlier	BOOLEAN	ระบุว่าวันนี้เป็นค่าผิดปกติหรือไม่ — เกณฑ์คือ $ \text{sales_diff} \geq 50$ เปอร์เซ็นต์	เกณฑ์คือ $ \text{sales_diff} \geq 50$ (ค่าคงที่ในโค้ด · เปลี่ยนต้องขออนุมัติธุรกิจ) · วัน outlier ถูกตัดออกจากการคำนวณ growth
source_checksum	VARCHAR(64)	checksum ของไฟล์ต้นทางที่แถวนี้มาจาก	ใช้กันนำเข้าไฟล์เดิมซ้ำ และพิสูจน์ว่าข้อมูลในตารางตรงกับไฟล์ที่ IAS ส่งมาจริง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ : เป็นฐานคำนวณอายุงานของ watchdog · ใช้ล๊ิบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ

sgi_fgi_impact_competitors

14 คอลัมน์

ร้านคู่แข่งที่กระทบ (ระดับ pipeline)

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_COMPETITOR (PK IMPACT_COMPETITOR_ID) · ข้อมูลใหม่ดึงจาก ALLMAP โดย Job 3
ใช้ทำอะไร	เก็บ สาขาของร้านคู่แข่ง ที่อยู่ใ้ในรัศมีกระทบของรอบนี้ ตามที่ ALLMAP ให้มา
ทำไมต้องมี	คู่แข่งเป็นข้อมูลประกอบการพิจารณา — ผู้อนุมัติต้องเห็นว่าคู่แข่งเปิดใหม่ในพื้นที่ด้วยหรือไม่ ก่อนตัดสินใจยอดตกเพราะร้าน 7-Eleven ด้วยกันจริง
ใครเขียน	Job 3 (ImportImpactCompetitor) — ดึงจาก ALLMAP
ใครอ่าน	Job 7 (sync เข้าเอกสารเป็น sgi_document_competitors)

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- แถวนี้เป็น ข้อมูลดิบฝั่ง pipeline · เมื่อ Job 7 sync เข้าเอกสารจะได้แถวใหม่ใน sgi_document_competitors ที่ผู้ใช้แก้ไขได้ — แก่ที่เอกสารไม่ย้อนกลับมาแก้ที่นี่

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มัดคู่แข่งเข้ากับรอบ
competitor_store_code	VARCHAR(50)	รหัสสาขาของคู่แข่งตามที่ ALLMAP ส่งมา เช่น 4832 · 789 · TD58_08	ไม่มี FK เพราะเป็นรหัสของระบบภายนอก · nullable เพราะ ALLMAP ส่งค่าว่างมาได้จริง (56 จาก 1,312 แถวในไฟล์ตัวอย่าง) · เป็นคีย์กันชำระระดับสาขา
brand_code	VARCHAR(30)	รหัสแบรนด์ 01–11	FK ไป sgi_competitors · nullable เพราะ map แบนด์ไม่ได้ก็ต้องเก็บแถวไว้ แล้วรายงานเป็น unmapped ไม่ใช่ทั้งข้อมูล
name_th	VARCHAR(200)	ชื่อร้านคู่แข่งภาษาไทยตามที่ ALLMAP ให้มา	เก็บค่าดิบไว้เทียบ เผื่อ match แบนด์ไม่ได้
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษที่ ALLMAP ส่งมา	ฟิลด์ 4 ของไฟล์ BPM06003O — ต้องส่งต่อ K2 จึงเก็บเป็น snapshot
branch_th	VARCHAR(200)	ชื่อสาขาของคู่แข่ง	ระบุว่าเป็นสาขาไหน — แบนด์เดียวกันมีได้หลายสาขาในรัศมี
zone_code	VARCHAR(10)	โซนของสาขาคู่แข่ง	ฟิลด์ 6 ของ BPM06003O — ส่งต่อ K2
subzone_code	VARCHAR(10)	โซนย่อยของสาขาคู่แข่ง	ฟิลด์ 7 ของ BPM06003O — ส่งต่อ K2
opened_date	DATE	วันที่สาขาคู่แข่งเปิด	ใช้ดูว่าเปิดก่อนหรือหลังร้าน 7-Eleven ใหม่ — มีผลต่อการตีความสาเหตุยอดตก

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
closed_date	DATE	วันที่สาขาคู่แข่งปิด (ถ้าปิดแล้ว)	สาขาที่ปิดไปแล้วไม่ควรนับเป็นปัจจัยกระทบของงวดหลังจากนั้น
period_key	CHAR(7)	งวด YYYY-MM ที่ข้อมูลชุดนี้เป็นของ	ใช้แยกข้อมูลคู่แข่งคนละงวดออกจากกัน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE
competitor_key	VARCHAR(300)	คีย์ธุรกิจของสาขาคู่แข่ง — generated: ใช้ competitor_store_code ถ้ามี ไม่งั้น fallback เป็น name:<ชื่อสาขา>	ALLMAP ส่งสาขาที่ไม่มีรหัสมาได้ (ORA COMPET_ID ไม่มี NOT NULL) · ใช้รหัสอย่างเดียวเป็นคีย์ไม่ได้: NULLS DISTINCT ทำให้แถวพอกทุกรอบ · NULLS NOT DISTINCT ยูนหลายสาขาเหลือแถวเดียว = ข้อมูลหาย (ร้าน 07109 มีแฟมิลีมาร์ท 80 สาขา) · คอลัมน์นี้จึงแยกสาขาด้วยชื่อเมื่อไม่มีรหัส

sgi_fgi_impact_compensations

18 คอลัมน์

ยอดชดเชยรายงวดของแต่ละรอบ

มาจากไหน	FGI/FCS · migrate จาก ORA FGI_IMPACT_STORE_COMPENSATE (PK COMPENSATE_I_ID) · รับเข้าโครงเมื่อ 2026-08-21 (ปิดช่องว่าง F1)
ใช้ทำอะไร	เก็บ ยอดเงินชดเชยรายงวด ของรอบหนึ่ง ทั้งค่าที่ระบบคาดการณ์และค่าที่ STA ปรับกลับมา
ทำไมต้องมี	รอบชดเชยหนึ่งรอบกินเวลาหลายเดือน (เดือน 1–3 และอาจหยุดที่เดือน 4) จึงต้องมีแถวแยกต่อ งวด ไม่ใช่เก็บยอดเดียวที่แถวแม่ · และเป็นตารางที่ Job 11 เขียนทับเมื่อ STA ส่งยอดจริงกลับมา
ใครเขียน	Job 6 (ตั้งยอดคาดการณ์) · Job 11 (ConsumeStaCompensate) — อัปเดต forecast_amount/adjust_amount จากข้อความ STA
ใครอ่าน	Job 6 (ส่งไป STA) · รายงาน · sgi_compensation_histories

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- compensate_status มีโดเมนปิด I/N/A/C/S/Z — C = ร้านปิดแล้ว หรือสัญญา SBP ถูกยกเลิก และตอนส่งไป STA ให้ส่งเป็น S (กลุ่มเดียวกับ Z)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
impact_process_id	BIGINT	FK ไปรอบชดเชย	มัดยอดเข้ากับรอบ
impacted_store_code	VARCHAR(5)	รหัสร้านที่ได้รับชดเชย	เก็บเข้าเพื่อ query รายงานโดยไม่ต้อง join
compensate_seq	INTEGER	ลำดับรอบชดเชย	บอกว่าเป็นการชดเชยครั้งที่เท่าไรของร้านนี้
compensate_seq_no	INTEGER	ลำดับย่อยภายในรอบ	ใช้เมื่อรอบเดียวมีหลายงวดย่อย
compensate_month	CHAR(7)	งวดเดือนที่ชดเชย (YYYY-MM ค.ศ.)	ระบุว่ายอดก่อนนี้เป็นของเดือนไหน
compensate_year	INTEGER	ปี ค.ศ. ของงวด	แยกไว้เพื่อ query รายปี

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
forecast_amount	NUMERIC(14,2)	ยอดชดเชยที่ระบบคำนวณคาดการณ์ไว้	ค่าที่ SGI ส่งไปให้ STA ตั้งต้น
adjust_amount	NUMERIC(14,2)	ยอดหลัง STA ปรับแก้	Job 11 เขียนค่านี้เมื่อได้ข้อความ sta_update_compensate — ยอดที่ใช้จ่ายจริงคือค่านี้เมื่อมีค่า
compensate_status	VARCHAR(5)	สถานะของยอดก่อนนี้ (I ตั้งต้น · N ไม่ชดเชย · A อนุมัติ · C ร้านปิด/ยกเลิกสัญญา · S/Z กลุ่มส่งออก)	ตัวควมว่ายอดนี้จะถูกส่งไป STA อย่างไร และจะนับเข้ารายงานหรือไม่
compensate_comment	VARCHAR(4000)	หมายเหตุประกอบยอด (ยาวได้ถึง 4000 ตัวอักษร)	เหตุผลที่ปรับยอดหรือไม่ชดเชย — ใช้ตอนตรวจสอบย้อนหลัง
stmt_month	INTEGER	เดือนของ statement ที่ยอดนี้ไปปรากฏ	ใช้กระทบยอดกับฝั่งบัญชี/Statement
stmt_year	INTEGER	ปีของ statement	คู่กับ stmt_month
approve_date	DATE	วันที่ยอดนี้ได้รับอนุมัติ	ใช้ในรายงานและการกระทบยอดกับบัญชี
created_by	VARCHAR(100)	ผู้สร้างแถว — x-user-id ของผู้ใช้จริง หรือชื่อ job (เช่น JOB-8) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ติดตั้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สืบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ
updated_by	VARCHAR(100)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_interface_transactions

26 คอลัมน์

สมุดกลางของการรับ-ส่งข้อมูลกับระบบภายนอก (tracking + transactional outbox)

มาจากไหน	ตารางใหม่ · แทน ORA FGI_CONFIRM_RECEIVE_DATA ซึ่งมีบัก 2 ข้อ: TRANSACTION_PK เป็น polymorphic FK และงาน purge ไม่ทำงานจริง
ใช้ทำอะไร	บันทึก ทุกครั้งที่ระบบรับหรือส่งข้อมูลกับระบบภายนอก (IAS · STA · การส่งต่อภายใน) 1 แถว = 1 record ที่รับส่ง · และทำหน้าที่เป็น transactional outbox ของข้อความ RabbitMQ
ทำไมต้องมี	สองเรื่องในตารางเดียว (ตั้งใจ): (1) ตามรอย ว่าอะไรส่งไปแล้ว/ยังไม่ถึง (2) กันข้อความหาย — Job 6 เขียนแถว READY ใน transaction เดียวกับการอัปเดตข้อมูล แล้วค่อย publish นอก transaction ถ้าระบบลงกลางทาง แถว READY ยังอยู่และส่งซ้ำได้
ใครเขียน	Job 4/5/6/8/9 (เขียน outbox + tracking) · Job 11 (บันทึกข้อความขาเข้า)
ใครอ่าน	Job 10 (watchdog ข้อความค้างส่ง) · GET /sgi/interface/tracking · /pending-ack

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- UNIQUE (data_name, direction, business_key, period_key) — กันส่ง/รับซ้ำ และเป็นกลไก idempotent หลักของทุก job
- ck_interface_typed_reference บังคับว่าต้องมีอย่างน้อย 1 ใน impact_process_id / sales_summary_id / doc_no — แก้นัก polymorphic FK ของระบบเดิม
- • มติ 2026-09-08 (ข้อ 2.13): ระบบ STA ไม่ตอบรับระดับธุรกิจกลับมา — ความสำเร็จของข่าอกวัดจาก outbox_status = 'CONFIRMED' (publisher confirm ของ RabbitMQ)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
run_id	VARCHAR(50)	รหัสรอบการรันที่สร้างแถวนี้ (มาจาก application log)	ใช้ผูกทุกแถวที่เกิดจากการรันครั้งเดียวกัน ตอนไล่หาสาเหตุ · ไม่มี FK เพราะตาราง job_run_histories ถูกตัดไปแล้ว
data_name	VARCHAR(80)	ชื่อชุดข้อมูลที่รับส่ง เช่น COMPENSATE_INIT_I · IMPACT_STORE_SALES (โตเมนปีด 12 ค่า)	เป็นส่วนหนึ่งของ UNIQUE ที่กันส่งซ้ำ และเป็นตัวกรองของ watchdog · พิมพ์ผิดตัวเดียว = กันซ้ำไม่ทำงาน + watchdog เจียน จึงต้องล็อกด้วย CHECK
direction	VARCHAR(10)	ทิศทาง: OUT ส่งออก · IN รับเข้า · INTERNAL ส่งต่อภายในระบบ	INTERNAL คือสิ่งที่มาแทนไฟล์ EAI เดิม (Jobs 7/8/9 เขียน DB ตรง) จึงจบที่ COMPLETED ทันที ไม่มีอะไรให้รอ · watchdog นับเฉพาะ OUT
status	VARCHAR(20)	วงจรชีวิตของแถว: READY → SENT → COMPLETED (หรือ FAILED/FAILED_RETRY)	สถานะระดับ *แถว* · ตัดค่า ACKED ออกเมื่อ 2026-09-08 เพราะไม่มี ACK จาก STA ให้รอ
impact_process_id	BIGINT	typed FK ไปรอบชุดเซช (ถ้าแถวนี้เกี่ยวกับรอบ)	หนึ่งใน 3 ทางอ้างอิงที่มาแทน polymorphic FK เดิม
sales_summary_id	BIGINT	typed FK ไปห่วยยอดขาย (ถ้าแถวนี้เกี่ยวกับยอดขาย)	ใช้กับ dataset ของ Job 4/5
doc_no	VARCHAR(10)	typed FK ไปเอกสาร (ถ้าแถวนี้เกี่ยวกับเอกสาร)	nullable เพราะบางข้อความเกิดก่อนมีเลขเอกสาร · ผูกด้วย impact_process_id แทนได้
business_key	VARCHAR(200)	กุญแจธุรกิจของ record ที่รับส่ง เช่น รหัสร้าน หรือ storecode_i:storecode_n	ส่วนหนึ่งของ UNIQUE ที่กันซ้ำ — เป็นตัวบอกว่า "ข้อความนี้พูดถึงของชิ้นไหน"

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
period_key	VARCHAR(20)	งวดของ record	อีกส่วนของ UNIQUE — ของชิ้นเดียวกันคนละงวดถือเป็นคนละข้อความ
correlation_id	VARCHAR(100)	รหัสสำหรับจับคู่ข้อความขาไป-ขากลับ	▲ job ต้องอ่านค่านี้จาก envelope เอง (มติ 2026-09-12 · job consume คิวเอง) — ถาดันทางไม่ส่งมาต้องยืนยันกับเจ้าของข้อความ
file_name	VARCHAR(255)	ชื่อไฟล์ต้นทาง/ปลายทาง (กรณี interface แบบไฟล์)	ใช้ตามรอยว่าข้อมูลมาจากไฟล์ไหนบน EAI S3
file_checksum	VARCHAR(64)	checksum ของไฟล์	พิสูจน์ว่าไฟล์ที่ประมวลผลคือไฟล์เดียวกับที่ต้นทางส่ง และกันประมวลผลไฟล์ซ้ำ
outbox_status	VARCHAR(20)	สถานะการ publish: READY รอส่ง · PUBLISHED ยิงแล้วรอ confirm · CONFIRMED broker ยืนยันแล้ว · FAILED ส่งไม่สำเร็จ	● เกณฑ์ตัดสินของ Job 10 ตั้งแต่มติ 2026-09-08 — แถว OUT ที่ไม่ใช่ CONFIRMED เกิน 1 วัน = ค้างส่ง ต้องส่งอีเมลเตือน
return_code	VARCHAR(50)	รหัสผลลัพธ์ที่ปลายทางตอบกลับ (ถ้ามี)	ใช้กับ interface ที่มีการตอบกลับ — ไม่ใช่เงื่อนไขของ watchdog แล้ว
return_message	VARCHAR(500)	ข้อความผลลัพธ์จากปลายทาง	ใช้อ่านตอนแก้ปัญหา
payload	JSONB	ข้อความที่ส่งออกไปจริง (JSONB)	เก็บของจริงไว้เพื่อส่งซ้ำได้ตรงตัว — ถ้าประกอบใหม่จากข้อมูลในฐานที่เปลี่ยนไปแล้ว ปลายทางจะได้คนละฉบับกับที่เคยส่งโดยไม่มีใครรู้ · เก็บเฉพาะขาออก
payload_version	SMALLINT	เวอร์ชันรูปแบบของ payload	ขึ้นเมื่อ contract กับปลายทางเปลี่ยน ไม่ใช่เมื่อข้อมูลเปลี่ยน — ใช้ตอนอ่าน payload เก้ากลับมาส่งซ้ำ
retry_count	INTEGER	จำนวนครั้งที่ลองส่งซ้ำ	ใช้ตัดสินว่าควรหยุดลองแล้วแจ้งคนหรือยัง
sent_at	TIMESTAMP	เวลาที่ publish ออกไป	ใช้วัดว่าค้างนานแค่ไหนหลังส่ง
acked_at	TIMESTAMP	เวลาที่ได้ publisher confirm จาก broker	ชื่อคอลัมน์คงเดิมเพื่อ compatibility แต่ความหมายเปลี่ยนตามมติ 2026-09-08 — ไม่ใช่เวลาที่ STA ตอบรับ
last_ack_notified_on	DATE	วันที่ watchdog ส่งอีเมลเตือนแถวนี้ล่าสุด	marker กันส่งอีเมลซ้ำในวันเดียวกัน ตอน Job 10 รันซ้ำ · ย้ายมาจากตาราง audit_logs ที่ถูกตัดเมื่อ 2026-08-07
purge_after	TIMESTAMP	วันที่แถวนี้นับทิ้งได้	ใช้โดย retention worker — ลบเฉพาะแถว COMPLETED ที่เลยกำหนดและไม่ติด legal hold
legal_hold	BOOLEAN	รงห้ามลบ (มีคดี/ตรวจสอบ)	กัน retention worker ลบหลักฐานที่ยังต้องใช้
created_at	TIMESTAMP	เวลาที่แถวนี้นี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้สับย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ
completed_at	TIMESTAMP	เวลาที่แถวนี้นี้จบวงจร	ใช้วัด lead time ของ interface และเป็นฐานของ purge_after

โซน B

เอกสารประกันรายได้ (workflow ใช้ engine กลาง) — 9 ตาราง

sgi_compensation_documents

26 คอลัมน์

เอกสารประกันรายได้ (หัวเอกสาร) — ตารางกลางของทั้งโซน B

มาจากไหน	K2 · migrate จาก MSSQL CompensateFlow (PK CompDocumentID → doc_no) join กับ ORA FGI_IMPACT_STORE_INFO เพื่อเติม impact_process_id
ใช้ทำอะไร	เก็บ เอกสารขอชดเชยหนึ่งฉบับ — สิ่งที่ใช้เห็น อนุมัติ และส่งต่อใน workflow 5 ขั้น
ทำไมต้องมี	เป็นจุดเชื่อมระหว่างโลก batch (โซน A) กับโลกคน (workflow อนุมัติ) · ทุกตารางลูกในโซน B ห้อยจาก doc_no ของตารางนี้ และ engine workflow อ้างเอกสารนี้ผ่าน referenceId
ใครเขียน	Job 8 (CreateCompensationDocument) · POST/PUT /sgi/document · POST /sgi/document/{docNo}/actions
ใครอ่าน	ทุกหน้าจอฝั่ง FE · รายงาน SBP Mail · Job 6 (ส่งยอดไป STA)

กติกาก่อนที่ตัวกรอกจะเขียนโค้ด

- ✓ มติ DP-1 (2026-08-10): PK เป็น surrogate id · doc_no เป็น UNIQUE ไม่ใช่ PK — referenceId ที่ส่งให้ @srm/glb-workflow คือ id (ส่งเป็น string เพราะ reference_id เป็น varchar(255))
- ▲ ตารางลูก 6 ตัวมี doc_no เป็น NOT NULL จึงต้องออกเลขเอกสารใน INSERT เดียวกันที่สร้างหัวเสมอ
- ไม่มีคอลัมน์ assignee — ผู้รับผิดชอบปัจจุบันอยู่ที่ engine (workflow_transaction.current_approver) ที่เดียว

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	เลขที่เอกสาร รูปแบบ YYYY/xxxxx โดย YYYY เป็น ค.ศ.	เลขที่ผู้ใช้เห็นและใช้อ้างอิงทุกที่ · เป็น UNIQUE และเป็นเป้าของ FK จากตารางลูก 8 ตัว · ▲ ต้อง encodeURIComponent เสมอเมื่อใส่ใน URL เพราะมี /
year	INTEGER	ปี ค.ศ. ของเอกสาร	คู่กับ running_no เพื่อประกอบเป็น doc_no และใช้กรอกรายปี
running_no	INTEGER	เลขลำดับในปีนั้น	ได้มาจาก sgi_document_running_numbers — เลขนี้ห้ามซ้ำในปีเดียวกัน
impact_process_id	BIGINT	FK ย้อนกลับไปรอบชดเชยในโซน A	เส้นเชื่อมสองโลก — ทำให้จากเอกสารย้อนไปดูยอดขาย/ร้านเปลี่ยนใหม่/คู่แข่งที่เป็นต้นเหตุได้
impact_compensation_id	BIGINT	แถวค่าชดเชยของงวดนั้น ที่เอกสารใบนี้ว่าด้วย (1:1)	ตัวตนจริงของเอกสาร — รอบชดเชยหนึ่งรอบกินหลายงวด และแต่ละงวดมีเอกสารของตัวเอง · ORA FGI_IMPACT_STORE_INFO 20,676 แถว ต่อค่าชดเชยรายงวด 20,674 แถว = 1.0001 · ใช้เป็นคีย์กันสร้างเอกสารซ้ำ (ON CONFLICT) แทน impact_process_id ที่เดิมผิด
impacted_store_code	VARCHAR(5)	รหัสร้านที่ขอชดเชย	ใช้แสดงหัวเอกสารและกรอกรายงานโดยไม่ต้อง join ขึ้นโซน A
impact_month	CHAR(7)	งวดเดือนที่ขอชดเชย (YYYY-MM ค.ศ.)	คู่กับรหัสร้านเป็น business key ของเอกสาร
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่หลักของเอกสารนี้	ใช้แสดงหัวเอกสาร — รายละเอียดครบบอยู่ที่ sgi_document_new_stores
round_no	INTEGER	รอบที่เท่าไรของการยื่นเรื่องนี้	รองรับกรณี เปิดเอกสารใหม่ได้เองหลังจบด้วยเหตุ/ไม่เห็นควรชดเชย (SDD GI ตัดค่าเปิด SR เดือนละ ~20,000 บาท)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
loop_no	INTEGER	รอบการวนกลับภายใน workflow	นับจำนวนครั้งที่เอกสารถูกส่งกลับมาแก้ — ใช้ดูว่าเอกสารวนนานผิดปกติไหม
source	VARCHAR(20)	ที่มาของเอกสาร: FS (สร้างจาก pipeline) หรือ MANUAL (คีย์มือ)	ใช้แยกว่าเอกสารมาจากงานอัตโนมัติหรือคน — มีผลต่อการตรวจสอบเมื่อข้อมูลไม่ตรง
status_code	VARCHAR(2)	รหัสสถานะเอกสาร (1 ใน 6 ค่า)	ตัวบ่งชี้ป้ายสถานะและตัวกรองหลักของหน้ารายการ/รายงาน · ค่าที่เป็นไปได้ตรงกับ workflow_status_document.md
current_section_code	VARCHAR(2)	รหัสชั้นที่เอกสารค้างอยู่ (06/08/01/02/03/99)	ใช้ตอบว่า "ตอนนี้ใครต้องทำ" · เป็นสำเนาไว้ query เร็ว — ค่าจริงที่ตัดสินสิทธิ์คือของ engine ผ่าน getTransaction()
total_compensation_amount	NUMERIC(14,2)	ยอดชดเชยรวมของเอกสาร (บาท)	ตัวตัดสินเส้นทางอนุมัติ: < 100,000 จบที่ GM (02) · ≥ 100,000 ไปต่อ AVP (03) (เกณฑ์เดียว มติ 2026-08-18)
allmap_url	VARCHAR(500)	ลิงก์แผนที่ ALLMAP ของร้านนี้	ป้อนเปิดแผนที่ในหน้าเอกสาร — เก็บ URL เพราะแผนที่เป็นระบบที่อื่น
statement_id	VARCHAR(50)	เลขอ้างอิง statement ฝั่งบัญชี	ใช้กระทบยอดกับระบบ Statement/STA
statement_date	DATE	วันที่ของ statement	คู่กับ statement_id
account_year	INTEGER	ปีบัญชีที่ยอดนี้ลง	ใช้ในรายงานฝั่งบัญชี
account_month	INTEGER	เดือนบัญชีที่ยอดนี้ลง	คู่กับ account_year — บัญชีใช้ค้นด้วยงวดบัญชี ไม่ใช่งวดกระทบ
approver_snapshot	JSONB	สำเนา ชื่อและตำแหน่งผู้อนุมัติ ณ เวลาที่อนุมัติ (JSONB)	เก็บเป็น snapshot เพราะคนย้ายตำแหน่ง/ลาออกได้ — เอกสารเก่าต้องยังพิมพ์ชื่อผู้อนุมัติเดิมได้ถูก · ▲ เป็น ข้อมูลส่วนบุคคล อยู่ในขอบเขต PDPA (ข้อค้าง 2.6)
version_no	INTEGER	เลขเวอร์ชันของแถว เพิ่มทีละ 1 ทุกครั้งที่แก้	optimistic locking — ถ้าสองคนเปิดเอกสารเดียวกันแล้วกดบันทึกพร้อมกัน คนที่สองจะได้ 409 STALE_VERSION แทนที่จะเขียนทับเจียน ๆ
created_by	VARCHAR(30)	ผู้สร้างแถว — x-user-id ของผู้ใช้จริง หรือชื่อ job (เช่น JOB-8) เมื่อ batch เป็นคนสร้าง	ใช้ตอบคำถาม "ใครทำให้เกิดแถวนี้" · แทนระบบ audit ที่ติดตั้งไปเมื่อ 2026-08-07 จึงเป็นร่องรอยชั้นเดียวที่เหลือ
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้ลีย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_document_new_stores

9 คอลัมน์

ร้านเปิดใหม่ในเอกสาร + การปันส่วน %ชดเชย

มาจากไหน	K2 · migrate จาก MSSQL ImpactProfile (ฝั่ง _N) + ORA FGI_NEW_STORE_INFO / FGI_NEW_STORE_COMPENSATE
ใช้ทำอะไร	เก็บว่าเอกสารฉบับนี้ขอชดเชยจาก ร้านเปิดใหม่ร้านไหนบ้าง ร้านละกี่ % และเป็นเงินเท่าไร
ทำไมต้องมี	เป็นตารางที่ผู้ใช้ แก่ได้จริงในหน้าจอ (ปรับ %) ต่างจาก sgi_fgi_impact_stores ที่เป็นข้อมูล pipeline · การแยกสองตารางทำให้แก้ไขเอกสารแล้วไม่ไปทับข้อมูลดิบของ batch
ใครเขียน	Job 9 (sync จากโซน A) · หน้าเอกสารผ่าน PUT /sgi/document/{docNo}
ใครอ่าน	หน้าเอกสาร · การคำนวณ total_compensation_amount · Job 6

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ผลรวม compensate_percent ของทุกแถวใน doc_no เดียวกันต้องเท่ากับ 100 พอดี — validate ก่อน commit

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดแฉกเข้ากับเอกสาร · ON DELETE CASCADE — ลบเอกสารแล้วแถวนี้หายตาม
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่	join ไป master ร้านของระบบเดิมเพื่อเอาชื่อ/ที่อยู่
distance_km	NUMERIC(8,3)	ระยะทางจากร้านเปิดใหม่ถึงร้าน SP	แสดงในตารางให้ผู้อนุมัติเห็นว่าใกล้แค่ไหน · ต้องอยู่ในรัศมีเกณฑ์
compensate_percent	NUMERIC(7,4)	%ชดเชยที่ปันให้ร้านนี้	ตัวคูณคำนวณ compensation_amount · รวมทุกแถวต้อง = 100
compensation_amount	NUMERIC(14,2)	ยอดเงินชดเชยของร้านนี้	= ยอดรวมเอกสาร × compensate_percent · รวมทุกแถว = total_compensation_amount
source_system	VARCHAR(30)	ที่มาของแถว: FGI (มาจาก pipeline) หรือ USER (ผู้ใช้เพิ่มเอง)	สำคัญมาก — ตอน Job 9 sync ใหม่จะ prune เฉพาะแถว FGI เท่านั้น แถวที่ผู้ใช้เพิ่มเองห้ามลบ
source_row_id	BIGINT	แถวต้นทางใน sgi_fgi_new_store_compensations ที่ Job 9 คัดลอกมา	%ชดเชยผิด = จ่ายเงินผิด จึงต้องตามรอยกลับได้ว่าตัวเลขมาจากแถวไหนของวดไหน · NULL = แถวที่ผู้ใช้เพิ่มเอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้ไขเมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_document_competitors

18 คอลัมน์

ร้านค้าแข่งในเอกสาร (ระดับสาขา)

มาจากไหน	K2 · migrate จาก MSSQL CompetInCompenProfile (+ ไฟล์ BPM060030_14 ฟิลด์)
ใช้ทำอะไร	เก็บ สาขาคู่แข่งที่ยกมาประกอบเอกสารฉบับนี้ พร้อมรายละเอียดและหมายเหตุที่ผู้ใช้เขียนเอง
ทำไมต้องมี	ผู้พิจารณาต้องเห็นว่าคู่แข่งเปิดใหม่ในพื้นที่ด้วยหรือไม่ · และต้อง เพิ่ม/แก้ได้เอง เพราะ ALLMAP ไม่ได้ครบเสมอ — จึงคัดลอกมาเป็นแถวของเอกสารแทนการอ่านจาก pipeline ตรง ๆ
ใครเขียน	Job 7 (sync จาก sgi_fgi_impact_competitors) · หน้าเอกสาร
ใครอ่าน	หน้าเอกสาร · รายงาน

กติกาก่อนกรอกข้อมูล

- ▲ คนละตารางกับ sgi_competitors — ที่นี่คือสาขา (หลายแถวต่อเอกสาร) ส่วนอีกตารางคือแบรนด์ (11 แถวทั้งระบบ)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแฉกเข้ากับเอกสาร
competitor_store_code	VARCHAR(50)	รหัสสาขาของคู่แข่ง (คัดลอกมาจาก Job 7)	ไม่มี FK · nullable เพราะแถวที่ผู้ใช้ขี้เกียจเองอาจไม่มีรหัสสาขา · เป็นคีย์กันข้าต่อเอกสาร
brand_code	VARCHAR(30)	รหัสแบรนด์ 01-11	FK ไป sgi_competitors — ทำให้สรุปรายงานรายแบรนด์ได้ · nullable เมื่อ map ไม่ได้
name_th	VARCHAR(200)	ชื่อร้านค้าแข่งภาษาไทย	แสดงในตารางของหน้าเอกสาร
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษ	ส่งต่อ K2 ในไฟล์ BPM060030
branch_th	VARCHAR(200)	ชื่อสาขา	ระบุสาขาที่กระทบ
zone_code	VARCHAR(10)	โซนของสาขาคู่แข่ง	ส่งต่อ K2
subzone_code	VARCHAR(10)	โซนย่อยของสาขาคู่แข่ง	ส่งต่อ K2
opened_date	DATE	วันที่สาขานี้เปิด	ใช้เทียบไทม์ไลน์กับวันที่ร้าน 7-Eleven ใหม่เปิด
closed_date	DATE	วันที่สาขานี้ปิด (ถ้ามี)	สาขาที่ปิดแล้วไม่ควรถูกนับเป็นเหตุกระทบของงวดถัดมา
impact_date	DATE	วันที่เริ่มกระทบตามที่ใช้ระบุ	ผู้ใช้ระบุเองได้เมื่อวันเปิดไม่ตรงกับวันที่เริ่มกระทบจริง
detail	TEXT	รายละเอียดที่ผู้ใช้พิมพ์	คำอธิบายประกอบการพิจารณา
remark	TEXT	หมายเหตุเพิ่มเติม	บันทึกช่วยจำของผู้ทำเรื่อง

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
source_system	VARCHAR(30)	ALLMAP (มาจาก pipeline) หรือ USER (ผู้ใช้เพิ่มเอง)	ตัวคุมว่าตอน Job 7 sync ใหม่จะ prune แกวไหนได้ — แกว USER ห้ามลบ
source_row_id	BIGINT	แถวต้นทางใน sgi_fgi_impact_competitors ที่ Job 7 คัดลอกมา	ตามรอยย้อนกลับตอน audit — ผู้พิจารณาที่กว่าข้อมูลคู่แข่งผิด ต้องรู้ว่ามาจากแถวไหน · ระบบเดิมส่ง PK ต้นทางเป็นฟิลด์ที่ 1 ของไฟล์ BPM06003O · NULL = แถวที่ผู้ใช้ตัวเอง (source_system = 'USER') ซึ่งไม่มีต้นทาง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE
competitor_key	VARCHAR(300)	คีย์ธุรกิจของสาขาคู่แข่ง — generated: ใช้ competitor_store_code ถ้ามี ไม่งั้น fallback เป็น name:<ชื่อสาขา>	ALLMAP ส่งสาขาที่ไม่มีรหัสมาได้ (ORA COMPET_ID ไม่มี NOT NULL) · ใช้รหัสอย่างเดียวเป็นคีย์ไม่ได้: NULLS DISTINCT ทำให้แถวพอกทุกรอบ · NULLS NOT DISTINCT ยบหลายสาขาเหลือแถวเดียว = ข้อมูลหาย (ร้าน 07109 มีแฟมิลีมาร์ท 80 สาขา) · คอลัมน์นี้จึงแยกสาขาด้วยชื่อเมื่อไม่มีรหัส

sgi_document_external_factors

8 คอลัมน์

ปัจจัยภายนอกที่ยกมาในเอกสาร

มาจากไหน	K2 · migrate จาก MSSQL FactorInCompenProfile
ใช้ทำอะไร	เก็บว่าเอกสารฉบับนี้อ้าง ปัจจัยภายนอกข้อไหน ช่วงวันที่เท่าไร เช่น ปิดถนนตั้งแต่วันไหนถึงวันไหน
ทำไมต้องมี	ปัจจัยภายนอกเป็นเหตุผลที่ทำให้ยอดตกโดยไม่เกี่ยวกับร้านเปิดใหม่ · ต้องมีช่วงวันที่เพื่อให้ผู้อนุมัติตัดสินใจว่าคาบเกี่ยวกับงวดที่ขอชดเชยจริงไหม
ใครเขียน	หน้าเอกสารผ่าน PUT /sgi/document/{docNo}
ใครอ่าน	หน้าเอกสาร · รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- date_to เป็น optional — ปัจจัยที่ยังไม่จบ (เช่น ก่อสร้างยังไม่เสร็จ) ปล่อยาวก็ได้

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแถวเข้ากับเอกสาร
factor_code	VARCHAR(30)	รหัสปัจจัยที่เลือกจาก master	FK ไป sgi_external_factors — บังคับให้เลือกจากรายการ ไม่ให้พิมพ์อิสระ
date_from	DATE	วันที่ปัจจัยเริ่มมีผล	ใช้ตรวจว่าคาบเกี่ยวกับงวดที่ขอชดเชยหรือไม่
date_to	DATE	วันที่ปัจจัยสิ้นสุด (ว่างได้ = ยังไม่จบ)	คู่กับ date_from เป็นช่วงเวลา

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
detail	TEXT	รายละเอียดของปัจจัยในบริบทของเอกสารนี้	อธิบายว่ากระทบบ้านนี้อย่างไร
remark	TEXT	หมายเหตุเพิ่มเติม	บันทึกช่วยจำ
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_consideration_logs

9 คอลัมน์

ประวัติการพิจารณาทุกชั้น (ใครตัดสินใจอะไร เมื่อไร)

มาจากไหน	K2 · migrate จาก MSSQL CompensateHistory (PK ActionID)
ใช้ทำอะไร	บันทึก ทุกครั้งที่มีคนกดปุ่มตัดสินใจ ในเอกสาร — ชั้นไหน ผลอะไร ใครทำ เมื่อไร พร้อมความเห็น
ทำไมต้องมี	engine @srm/glb-workflow เก็บประวัติของตัวเองก็จริง แต่ ไม่มีที่เก็บรหัสผลการพิจารณา และความเห็นแบบที่ธุรกิจนี้ต้องการ — จึงเป็นหนึ่งใน 3 ตารางที่เก็บไว้เองแม้จะดูซ้ำกับ engine
ใครเขียน	POST /sgi/document/{docNo}/actions — เขียนใน transaction เดียวกับการ trigger event ของ engine
ใครอ่าน	ไทม์ไลน์ในหน้าเอกสาร · รายงานตรวจสอบ

กติกาสี่ข้อก่อนเขียนโค้ด

- append-only — ห้าม UPDATE/DELETE แถวเก่า ประวัติต้องเรียงตามจริง
- ต้องเขียนใน transaction เดียวกับการเปลี่ยนสถานะเอกสาร ไม่จ้นประวัติกับสถานะจะไม่ตรงกัน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสรุ่นแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดประวัติเข้ากับเอกสาร · ไม่ CASCADE — ประวัติต้องอยู่แม้เอกสารถูกลบ
section_code	VARCHAR(2)	ชั้นที่เกิดการตัดสินใจ (06/08/01/02/03)	บอกว่าใครในสายอนุมัติเป็นคนตัดสินใจ
result	VARCHAR(100)	ผลการพิจารณาเป็นข้อความไทยตามที่ผู้ใช้เห็น (7 ค่า)	แสดงบนไทม์ไลน์ตรงตัว · ข้อความ verbatim จาก SRS ห้ามแก้คำ · ค่าที่ 7 คือ "คำนวณเงินชดเชยเรียบร้อยแล้ว" เพิ่มเมื่อ 2026-09-01
result_category	VARCHAR(50)	หมวดของผล: APPROVE · REJECT · PENDING · CANCELLED	ใช้จัดกลุ่มในรายงานโดยไม่ต้อง match ข้อความไทย ซึ่งเปลี่ยนค่าได้
detail	TEXT	ความเห็น/เหตุผลที่ผู้พิจารณาพิมพ์	หลักฐานประกอบการตัดสินใจ — ใช้ตอนตรวจสอบย้อนหลัง
consider_by	VARCHAR(30)	ผู้พิจารณา (x-user-id)	ตอบว่าใครเป็นคนตัดสินใจ · ▲ เป็นข้อมูลส่วนบุคคลตาม PDPA
action_datetime	TIMESTAMP	เวลาที่ตัดสินใจ	เรียงไทม์ไลน์ และใช้คำนวณว่างานค้างที่ชั้นไหนนานเท่าไร (Job 12)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
request_id	VARCHAR(80)	รหัสคำขอของ action นี้	idempotency key — ถ้าผู้ใช้กดปุ่มซ้ำหรือ retry เข้ามาด้วย request_id เดิม ต้องไม่เกิดประวัติซ้ำและไม่เลื่อน workflow สองครั้ง

sgi_document_attachments

16 คอลัมน์

ไฟล์แนบของเอกสาร (เก็บเฉพาะ metadata ตัวไฟล์อยู่บน S3)

มาจากไหน	K2 · migrate จาก MSSQL CompDocAttachment + CompTempAttachment (+ AttachFileProfile)
ใช้ทำอะไร	เก็บ ข้อมูลของไฟล์แนบ — ชื่อ ขนาด ชนิด ที่อยู่บน S3 และผลสแกนไวรัส
ทำไมต้องมี	ตัวไฟล์จริงอยู่บน S3 ผ่านบริการของระบบ SBP เดิม (POST /statement/upload-file-aws) · แต่ต้องมีตารางฝั่งเราเพื่อผูกไฟล์กับเอกสารและ ขั้นตอนที่แนบ ซึ่ง S3 ไม่รู้
ใครเขียน	POST /sgi/document/{docNo}/attachments
ใครอ่าน	หน้าเอกสาร · GET .../download และ /download-all

กติกาส่ที่ต้องรู้ก่อนเขียนโค้ด

- ไฟล์ละไม่เกิน 5 MB (บังคับด้วย CHECK (file_size <= 5242880))
- นโยบาย AV (2026-09-07): CLEAN ดาวน์โหลดได้เสมอ · BLOCKED/FAILED คืบ 422 เสมอ · PENDING ขึ้นกับสวิตช์ SGI_ALLOW_PENDING_DOWNLOAD ใน mas_param — ย้งรอ security sign-off (ข้อ 2.10)
- ทุก query ต้องกรอง deleted_flag = 'N' เสมอ

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
attach_id	BIGSERIAL	PK ของไฟล์แนบ	ใช้ใน URL ดาวน์โหลตรายไฟล์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL)	มัดไฟล์เข้ากับเอกสาร · ไม่ CASCADE เพราะไฟล์เป็นหลักฐาน
section_code	VARCHAR(2)	ชั้นที่ไฟล์นี้ถูกแนบ	ทำให้รู้ว่าไฟล์มาจากชั้นไหน และคุณลึธิว่าใครแนบ/ลบได้
file_name	VARCHAR(255)	ชื่อไฟล์ตามที่ใช้อัปโหลด	แสดงในตารางและใช้เป็นชื่อไฟล์ตอนดาวน์โหลด
mime_type	VARCHAR(100)	ชนิดไฟล์ เช่น application/pdf	ใช้ตั้ง Content-Type ตอนส่งไฟล์กลับ และกรองชนิดที่ไม่อนุญาต (415)
file_size	BIGINT	ขนาดไฟล์เป็นไบต์	บังคับเพดาน 5 MB ที่ระดับฐานข้อมูล — กันไฟล์ใหญ่หลุดเข้ามาแม่แอปพลาด
storage_provider	VARCHAR(30)	ผู้ให้บริการที่เก็บไฟล์ (เช่น S3)	เผื่อย้ายที่เก็บในอนาคตโดยไม่ต้องแก้โครง
bucket	VARCHAR(120)	ชื่อ bucket ที่เก็บไฟล์	ครึ่งหนึ่งของที่อยู่ไฟล์
object_key	VARCHAR(500)	path ของไฟล์ใน bucket	อีกครึ่ง — ใช้ประกอบกับ bucket เพื่อตั้งไฟล์จริง
sha256	VARCHAR(64)	ลายนิ้วมือของไฟล์	ใช้กันอัปโหลดไฟล์เดิมซ้ำ (อยู่ใน UNIQUE doc_no + sha256 + deleted_flag) และพิสูจน์ว่าไฟล์ไม่ถูกแก้

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
scan_status	VARCHAR(20)	ผลสแกนไวรัส: PENDING · CLEAN · BLOCKED · FAILED	ตัวคูณสิทธิ์ดาวน์โหลด ตามนโยบาย AV · ▲ ระบบยังไม่มีตัวสแกนจริง ค่าจะค้างที่ PENDING
scanned_at	TIMESTAMP	เวลาที่สแกนเสร็จ	ใช้ดูว่าผลสแกนเก่าแค่ไหน
scan_message	VARCHAR(500)	ข้อความผลสแกน	บอกเหตุผลเมื่อถูก block
uploaded_by	VARCHAR(30)	ผู้อัปโหลด	ตอบว่าใครเอาไฟล์เข้ามา
uploaded_at	TIMESTAMP	เวลาที่อัปโหลด	เรียงลำดับไฟล์และใช้ตรวจสอบย้อนหลัง
deleted_flag	CHAR(1)	ธงลบแบบ soft delete — CHAR(1) ค่า 'Y'/'N' (ไม่ใช่ boolean)	ลบแบบไม่ลบจริงเพื่อรักษาหลักฐาน · ทุก query ต้องกรอง = 'N' · ▲ ปัจจุบันยังไม่มี endpoint หรือปุ่มไหนตั้งค่าเป็น 'Y' — อยู่ในข้อค้าง 2.5

sgi_compensation_histories

8 คอลัมน์

ประวัติการชดเชยรายร้าน (ข้ามเอกสาร)

มาจากไหน	K2 · migrate จาก ORA FGI_IMPACT_STORE_COMPENSATE + MSSQL CompensateFlow (แฉรวบก่อนหน้าของร้านเดียวกัน)
ใช้ทำอะไร	เก็บ ประวัติว่าร้านนี้เคยได้ชดเชยงวดไหน เท่าไร ส่งบัญชีเดือนไหน โดยไม่ผูกกับเอกสารฉบับใดฉบับหนึ่ง
ทำไมต้องมี	ผู้พิจารณาต้องเห็นประวัติย้อนหลังของร้านก่อนอนุมัติงวดใหม่ · และกฎ "ยอด 0 ติดกัน 3 เดือน → เดือนที่ 4 หยุดชดเชย" ต้องนับจากประวัตินี้
ใครเขียน	Job 6 / Job 8 หลังยอดถูกอนุมัติ · migration ตอน cutover
ใครอ่าน	ส่วน "ประวัติการชดเชย" ในหน้าเอกสาร · รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- ref_doc_no เป็น nullable เพราะประวัติที่ย้ายมาจากระบบเดิมบางแถวไม่มีเอกสารคู่กัน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช้ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
store_code	VARCHAR(5)	รหัสร้านที่ประวัตินี้เป็นของ	แกนหลักในการดึงประวัติ — query ด้วยรหัสร้านไม่ใช่เลขเอกสาร
ref_doc_no	VARCHAR(10)	เลขเอกสารที่ทำให้เกิดยอดนี้ (ว่างได้)	ลิงก์กลับไปเอกสารต้นเรื่องเมื่อมี
submit_account_month	CHAR(7)	เดือนที่ส่งเข้าบัญชี (YYYY-MM)	ใช้กระทบยอดกับฝั่งบัญชีและเรียงประวัติ
compensate_amount	NUMERIC(14,2)	ยอดเงินที่ชดเชยงวดนั้น	ยอด 0 มีความหมาย — เป็นตัวนับว่าครบ 3 เดือนติดหรือยังตามกฎหมายหยุดชดเชย
accounting_status	VARCHAR(30)	สถานะฝั่งบัญชีของยอดนี้	บอกว่ายอดนี้ไปถึงขั้นไหนของบัญชีแล้ว

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
external_ref	VARCHAR(100)	เลขอ้างอิงของระบบภายนอก (STA/Statement)	ใช้ตามยอดข้ามระบบเมื่อกระทบยอดไม่ตรง
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ · เป็นฐานคำนวณอายุงานของ watchdog · ใช้ล๊ิบย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ

sgi_document_cost_details

10 คอลัมน์

ยอดชดเชยแยกรายเดือน × รายร้านเปิดใหม่

มาจากไหน	K2 · migrate จาก MSSQL ImpactCostDetail (PK ImpCostID) · เพิ่มเข้าโครง 2026-08-06
ใช้ทำอะไร	เก็บ ยอดเป้าหมายและยอดจริงแยกตามเดือนและร้านเปิดใหม่ สำหรับกลุ่ม N และ NC
ทำไมต้องมี	ยอดรวมในหัวเอกสารตอบโต้แค่ "เท่าไร" แต่ตอบไม่ได้ว่า "เดือนไหน ร้านไหน เท่าไร" · ผังบัญชีและการตรวจสอบต้องการรายละเอียดระดับนี้
ใครเขียน	Job 8 ตอนสร้างเอกสาร · การคำนวณยอดในหน้าเอกสาร
ใครอ่าน	ตารางรายละเอียดค่าใช้จ่ายในหน้าเอกสาร · รายงานบัญชี

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- 1 แถว = 1 (เอกสาร × ร้านเปิดใหม่ × ปี × เดือน)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
id	BIGSERIAL	กุญแจแทน (surrogate key) ที่ฐานข้อมูลออกให้เอง เรียงเพิ่มทีละ 1	ใช้เป็น PK และเป็นเป้าของ FK จากตารางลูก · ห้ามเอาไปโชว์ผู้ใช้ (ผู้ใช้เห็น doc_no หรือรหัสร้านแทน) · เหตุผลที่ไม่ใช่ business key เป็น PK คือ business key ของระบบเดิมเปลี่ยนได้และบางตารางมีคีย์ผสม 3-4 คอลัมน์
doc_no	VARCHAR(10)	FK ไปหัวเอกสาร (NOT NULL · CASCADE)	มัดแถวเข้ากับเอกสาร
new_store_code	VARCHAR(5)	รหัสร้านเปิดใหม่ที่ยอดก้อนนี้เป็นของ	แยกยอดรายร้าน
cost_year	SMALLINT	ปี ค.ศ. ของยอดก้อนนี้	แกนเวลาแรก
cost_month	SMALLINT	เดือนของยอดก้อนนี้ (1–12)	แกนเวลาที่สอง — ทำให้ดูยอดรายเดือนได้
cost_target_n	NUMERIC(14,2)	ยอด เป้าหมาย กลุ่ม N	ฐานเปรียบเทียบของกลุ่ม N
cost_amount_n	NUMERIC(14,2)	ยอด จริง กลุ่ม N	ค่าที่เกิดขึ้นจริง เทียบกับเป้าหมาย
cost_target_nc	NUMERIC(14,2)	ยอด เป้าหมาย กลุ่ม NC	ฐานเปรียบเทียบของกลุ่ม NC
cost_amount_nc	NUMERIC(14,2)	ยอด จริง กลุ่ม NC	ค่าที่เกิดขึ้นจริงของกลุ่ม NC

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
created_at	TIMESTAMP	เวลาที่แถวนี้ถูกสร้าง (ตั้งอัตโนมัติด้วย DEFAULT CURRENT_TIMESTAMP)	ใช้เรียงลำดับเหตุการณ์ : เป็นฐานคำนวณอายุงานของ watchdog · ใช้ลีย้อนตอน incident ว่าแถวเกิดตอนไหน — ห้ามให้อัปเขียนทับ

sgi_document_running_numbers

4 คอลัมน์

ตัวนับเลขเอกสารต่อปี

มาจากไหน	K2 · migrate จาก MSSQL RunningNumber · เพิ่มเข้าโครง 2026-08-06
ใช้ทำอะไร	เก็บ เลขลำดับล่าสุดของเอกสารในแต่ละปี เพื่อออกเลข doc_no ถัดไป
ทำไมต้องมี	เลขเอกสารต้องเรียงต่อกัน ไม่ข้ามและไม่ซ้ำภายในปีเดียว · ถ้าใช้ MAX(running_no) + 1 จะเกิดเลขซ้ำทันทีเมื่อมีสองคนสร้างพร้อมกัน ตารางนี้ทำให้ล็อกแถวเดียว (SELECT ... FOR UPDATE) แล้วออกเลขได้อย่างปลอดภัย
ใครเขียน	Job 8 และ POST /sgi/document ตอนออกเลขเอกสาร
ใครอ่าน	เฉพาะตอนออกเลข — ไม่มีหน้าจอไหนอ่าน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- PK เป็น year — 1 แถวต่อ 1 ปี · ต้องล็อกแถวก่อนบวกเลขเสมอ
- ปีเป็น ค.ศ. ตามมติ 2026-08-06 (ทั้งระบบใช้ ค.ศ. ยกเว้นไฟล์ interface)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
year	SMALLINT	ปี ค.ศ. ที่ตัวนับนี้เป็นของ	PK · ขึ้นปีใหม่ = แถวใหม่ เริ่มนับ 1 ใหม่
last_running_no	INTEGER	เลขลำดับล่าสุดที่ออกไปแล้วในปีนั้น	บวก 1 แล้วเอาไปประกอบเป็น doc_no = year/LPAD(no, 5, '0')
updated_by	VARCHAR(30)	ผู้แก้ไขล่าสุด (รูปแบบเดียวกับ created_by)	ใช้ตอบคำถาม "ใครแก้ล่าสุด" ตอนข้อมูลเพี้ยน
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

โซน C Master ที่ SGI เป็นเจ้าของเอง — 3 ตาราง

sgi_impacted_stores

8 คอลัมน์

ทะเบียนร้านที่ถูกกระทบ (ร้าน SP ที่อยู่ในขอบเขตประกันรายได้)

มาจากไหน	K2 · migrate จาก ORA FGI_IMPACT_STORE (เลือกเฉพาะฝั่ง _I แบบ distinct) รวมกับ MSSQL CompensateFlow.CompTransferSBPDate
ใช้ทำอะไร	เก็บ รายชื่อร้านที่อยู่ในข่ายได้รับการชดเชย พร้อมพิกัดและผู้ดูแล (DV) เป็นสะพานเชื่อมสองระบบ: store_code ที่นี้คือค่าเดียวกับ impacted_store_code ทั้งโซน A และ B
ทำไมต้องมี	ตาราง store/mas_store ของระบบ SBP เดิมมีร้าน ทุกประเภท ไม่ได้บอกว่าร้านไหนเป็น SP ที่อยู่ในโครงการประกันรายได้ · ถ้าไม่มีตารางนี้ ทุก query ต้องไปไล่เงื่อนไข SP เองทุกครั้ง และไม่มีที่เก็บพิกัดที่ใช่ว่าหมด ALLMAP กับวันโอนเข้า SBP
ใครเขียน	Job 2 (ImportImpactStore) — upsert จาก ALLMAP · migration ตอน cutover
ใครอ่าน	Job 4/5/6/8 · GET /sgi/document/{docNo} (สมุดแผนที่) · หน้ารายการ/รายงาน

กติกาที่ต้องรู้ก่อนเขียนโค้ด

- PK เป็น store_code ตรง ๆ ไม่ใช่ surrogate — เพราะรหัสร้าน 5 หลักของ 7-Eleven เป็นค่าคงที่ตลอดอายุร้าน และทุกระบบอ้างอิงด้วยรหัสนี้
- ไม่เก็บชื่อร้าน/ที่อยู่ซ้ำ — อ่านจาก store/mas_store ของระบบเดิมเสมอ เก็บซ้ำเมื่อไรข้อมูลจะเพี้ยนกันทันที

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
store_code	VARCHAR(5)	รหัสร้าน 5 หลักของร้านที่ถูกกระทบ (ร้าน SP)	PK · ใช้ join ไป store/mas_store ของระบบเดิมเพื่อเอาชื่อ/ที่อยู่ · เป็นค่าเดียวกับ impacted_store_code ในโซน A/B
dv_code	VARCHAR(20)	รหัสเขต (District Vendor) ที่ร้านนี้สังกัด	ใช้จัดกลุ่มรายงานรายเขต และเป็นตัวกรองของอีเมลสรุปราย DV (EM-06)
opt_dv_user_id	VARCHAR(30)	รหัสผู้ใช้ของ DV ที่ดูแลร้านนี้	ใช้เป็น เงื่อนไข gate ของ Job 8b: ถ้าว่าง = เปิด workflow ไม่ได้ เพราะไม่รู้จะมอบงานให้ใคร (ตั้ง workflow_generation_status = 'N')
latitude	NUMERIC(10,7)	ละติจูดของร้าน (ทศนิยม 7 ตำแหน่ง)	ใช้วาดหมุดบนแผนที่ ALLMAP ในหน้าเอกสาร · ความละเอียด 7 ตำแหน่ง ≈ 1 ซม. เพียงพอสำหรับรัศมี 1–2 กม.
longitude	NUMERIC(10,7)	ลองจิจูดของร้าน	คู่กับ latitude — ใช้คำนวณ/ตรวจระยะรัศมีกระทบ 1 กม. (กทม.) / 2 กม. (ต่างจังหวัด)
transfer_sbp_date	DATE	วันที่ร้านโอนเข้าเป็นร้าน SBP	ใช้ตัดสินใจว่าร้าน อยู่น้อยกว่า 12 เดือน 15 วันหรือไม่ ซึ่งเป็นเงื่อนไข pre-accept ของ Job 5 · nullable เพราะร้านเก่าบางร้านไม่มีวันที่นี้ในระบบเดิม
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างอิงรหัสนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง is_active = TRUE แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_external_factors

5 คอลัมน์

master ปัจจัยภายนอก

มาจากไหน	K2 · migrate จาก MSSQL FactorProfile — หน้าจอ k2-factors.html (SRS 3.1.9)
ใช้ทำอะไร	เก็บ รายการปัจจัยภายนอกที่ใช้อธิบายว่าทำไมยอดขายตก เช่น ก่อสร้าง/ปิดถนน/น้ำท่วม ให้ผู้ใช้เลือกจาก dropdown แทนการพิมพ์เอง
ทำไมต้องมี	ถ้าให้พิมพ์เป็นข้อความอิสระ จะสรุปรายงานว่า "ปัจจัยไหนทำให้ขาดขายบ่อที่สุด" ไม่ได้เลย เพราะคนละคนพิมพ์คนละแบบ
ใครเขียน	หน้า k2-factors.html ผ่าน POST/PUT/DELETE /sgi/master/factors
ใครอ่าน	dropdown ในหน้าเอกสาร · sgi_document_external_factors อ้างด้วย FK

กติกาก่อนที่ควรกรอกข้อมูล

- ลบจริงไม่ได้ถ้ามีเอกสารอ้างอิง — ใช้ is_active = FALSE แทน

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
factor_code	VARCHAR(30)	รหัสปัจจัย เช่น F003	PK · เป็นค่าที่ sgi_document_external_factors.factor_code อ้างถึง
factor_name	VARCHAR(200)	ชื่อปัจจัยภาษาไทยที่ผู้ใช้เห็น เช่น "การก่อสร้าง / ปิดถนน"	แสดงใน dropdown และในรายงาน
factor_remark	VARCHAR(500)	คำอธิบายเพิ่มเติมของปัจจัย	ช่วยให้ผู้ใช้เลือกถูกตัว — ไม่ได้ใช้ในการคำนวณใด ๆ
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างอิงนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง is_active = TRUE แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

sgi_competitors

6 คอลัมน์

master แบนด์คู่แข่ง

มาจากไหน	K2 · หน้าจอ k2-competitors.html (คัดลอกจากหน้าจอ K2 เดิมแบบตรงตัว 11 แบนด์ รหัส 01–11)
ใช้ทำอะไร	เก็บ รายชื่อแบรนด์ร้านค้าคู่แข่ง (Lotus's Express · CJ Express · ฯลฯ) ทั้งภาษาไทยและอังกฤษ
ทำไมต้องมี	ข้อมูลคู่แข่งที่ดึงจาก ALLMAP มาเป็น รายสาขา (มีชื่อสาขา พิกัด วันเปิด) แต่ต้องรู้ว่าสาขานั้นเป็นแบรนด์อะไรเพื่อสรุปรายงาน ตารางนี้คือชั้นแบรนด์
ใครเขียน	หน้า k2-competitors.html ผ่าน POST/PUT/DELETE /sgi/master/competitors
ใครอ่าน	dropdown "ร้านค้าคู่แข่ง (Master)" ในหน้าเอกสาร · sgi_document_competitors · sgi_fgi_impact_competitors

กติกาก่อนที่ควรรู้ก่อนเขียนโค้ด

- อย่าสับสนกับ sgi_document_competitors — ตารางนี้คือ *แบรนด์* (11 แถว) ส่วนอีกตารางคือ *สาขาของคู่แข่งที่กระทบเอกสารฉบับหนึ่ง* (หลายแถวต่อเอกสาร)

คอลัมน์	ชนิด	เก็บอะไร	ใช้ทำอะไร
competitor_code	VARCHAR(30)	รหัสแบรนด์ 01–11	PK · ถูกอ้างเป็น brand_code ในตารางคู่แข่งระดับสาขาทั้งสองตัว — คนละรหัสกับ competitor_store_code ที่ ALLMAP ส่งมา
name_th	VARCHAR(200)	ชื่อแบรนด์ภาษาไทย	แสดงในหน้าจอและรายงานทั้งหมด
name_en	VARCHAR(200)	ชื่อแบรนด์ภาษาอังกฤษ	ใช้ตอน export และตอน match ชื่อที่ได้จาก ALLMAP ซึ่งมาเป็นอังกฤษ
remark	VARCHAR(500)	หมายเหตุของแบรนด์	บันทึกช่วยจำ ไม่มีผลต่อ logic
is_active	BOOLEAN	ธงเปิด/ปิดการใช้งานของแถว master	ปิดแทนการลบ เพราะเอกสารเก่าที่อ้างรหัสนี้ต้องยังแสดงชื่อได้ · dropdown ฝั่ง FE ต้องกรอง is_active = TRUE แต่หน้าอ่านเอกสารเก่าห้ามกรอง
updated_at	TIMESTAMP	เวลาที่แถวนี้ถูกแก้ล่าสุด	ใช้ตรวจว่าข้อมูล master ถูกแก้เมื่อไร และใช้เป็นตัวช่วยตอน sync/รีเฟรช cache ฝั่ง FE

สรุปความครบถ้วน

ตารางที่อธิบายไว้	20 ตาราง — ครบทุกตารางที่ SGI ต้องสร้างใหม่
คอลัมน์ที่อธิบายไว้	251 คอลัมน์ — ครบทุกคอลัมน์ใน DDL
คอลัมน์ที่ยังไม่มีคำอธิบาย	0 — กฎ #88 ใน tools/check_docs.py บังคับให้เป็น 0
เอกสารนี้ตกยุคได้ไหม	ไม่ได้ — รายชื่อคอลัมน์อ่านสดจาก DDL ถ้า schema เพิ่มคอลัมน์แล้วยังไม่มีคำอธิบาย ชุดตรวจจะไม่ผ่านทันที